COPILOT STUDIO · POSITION & PROMISE

The studio that turns business intent into agents.

Microsoft Copilot Studio is the low-code platform for building conversational agents, autonomous agents, and agent flows that act on enterprise data — without standing up a model of your own.

It sits between your **people & channels**, the **foundation model**, and your **enterprise data** — orchestrating retrieval, instructions, tools, and governance at the seam between prompt and outcome.

WHY IT EXISTS

Makes generative AI a first-class business surface — secure, governed, and observable.

Not a chatbot kit. Not a prompt playground. A **runtime, control plane, and authoring canvas**, together.

SURFACE
Channels & users

AUTHORING + RUNTIME
Microsoft Copilot Studio
Agents · Agent Flows · Topics

REASONING
Foundation model

GROUND TRUTH
Knowledge & actions

STACK VIEW

User request is decomposed, retrieved, and answered in one governed path.

AGENT CONCEPTS · ANATOMY

An agent is a shape, not a script.

A **Copilot Studio agent** is a configured bundle of identity + knowledge + actions + conversational routes, governed as a unit, executed by an orchestrator against a chosen model.

01 · KNOWLEDGE

Where answers come from

SharePoint, Dataverse, public sites, files, Azure AI Search.

02 · TOPICS

Conversational routes

Trigger phrases → branched flows with conditions.

IDENTITY

Agent

name · description · instructions

04 · TRIGGERS

What wakes the agent

User utterance, event, schedule, or a parent agent's call.

05 · CHANNELS

Where it shows up

Microsoft 365 Copilot, Teams, web, custom Direct Line.

06 · ORCHESTRATOR

The reasoning layer

Plans, calls tools, watches for completion, decides next step.

03 · ACTIONS

Tools it can call

Pre-built connectors, custom connectors, prompts, agent flows.

WHAT AN AGENT IS NOT

— a static FAQ list

— a single prompt

— a model you bring

It is a runtime object with a lifecycle: created, configured, published, analysed, versioned.

Each capability is independently configured — agents compose them like Lego, and the same identity can be re-tuned without touching the wiring.

FOUNDATION MODELS · PRIMER

What a foundation model actually is.

A **foundation model** is a large neural network pre-trained on broad text (and often code, image, audio) at internet scale, then **adapted** — not rewritten — for your task.

In Copilot Studio you do not pick parameters — you pick the model and let the studio handle host, prompts, and grounding. The model is the engine; your instructions, knowledge, and tools are the cage, the road, and the cargo.

PRE-TRAINING

Self-supervised at scale

Trillions of tokens. Patterns, not facts, are the residue.

ADAPT

Tuned for reasoning

Instruction tuning + RLHF. The base is steered, not replaced.

GROUND

Retrieval augments it

At inference, your enterprise data joins the prompt — facts first.

ANSWER

Tokens out, grounded

Generation is the final step — most of the work happened earlier.

MENTAL MODEL

A model that knows the world. Your task is to teach it your world.

Pick a model for its strengths — reasoning depth, latency, cost — then guide it with instructions and knowledge. The model is not the agent; the agent is what you build on top.

TOKENS & CONTEXT WINDOWS

The model reads in tokens, not paragraphs.

A **token** is the smallest chunk of text the model actually sees — typically a word piece, a digit, or a punctuation symbol. A **context window** is the model's working memory: the maximum number of tokens it can hold at once, every turn, every call.

Rough rule: 1 token ≈ 4 characters of English, ¾ of a word. The same window can hold a contract or forget a five-turn conversation — depending on how you spend it.

ANATOMY OF A CONTEXT WINDOW

Order matters. Earlier tokens survive compression; later tokens get squeezed out.



One call, end-to-end. Everything that fits, the model sees. Everything that does not, the model cannot.

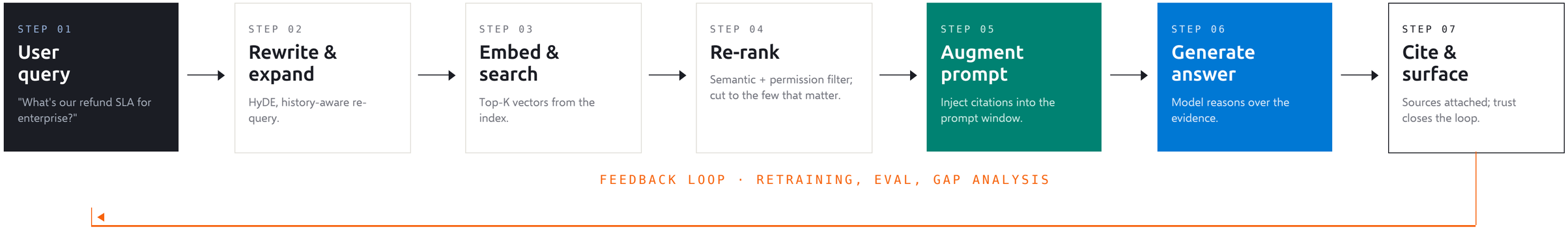
RULES WORTH MEMORISING

1. **Cost scales with tokens in and tokens out.**
2. **Bigger knowledge ≠ better answers. Retrieve narrowly.**
3. **Past turns steal budget from the next answer.**
4. **Compression is lossy. Older turns go first.**
5. **Output tokens are scarce — answer fully, briefly.**

FOUNDATION MODELS · RAG

Retrieval-Augmented Generation. Don't make the model remember — let it look.

RAG is the pattern where, before any answer is generated, the agent retrieves relevant passages from enterprise sources and feeds them back into the prompt. The model reasons; the documents prove. The retrieval is the agent's working memory.



WHY IT MATTERS

Without RAG the model hallucinates. Without citations the user cannot verify. Without permission filtering the agent leaks. RAG is what makes an agent trustworthy at work.

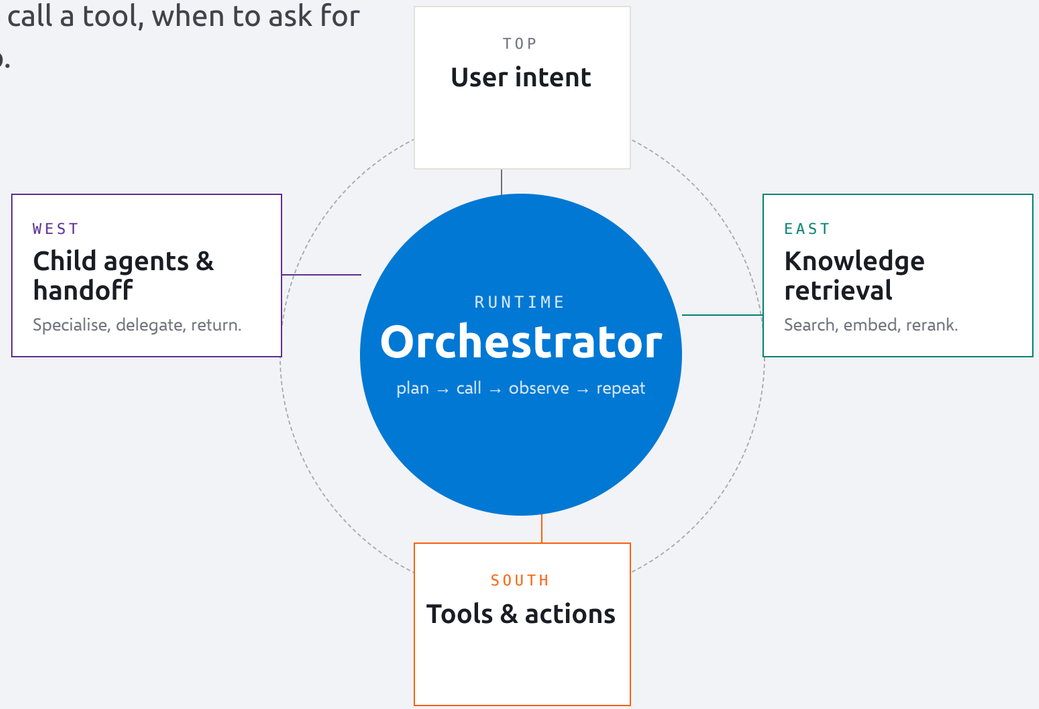
IN COPILOT STUDIO

Knowledge sources list, search mode (semantic / hybrid / classic), result count, citation style — all surfaced as authoring controls. The agent decides when to retrieve.

FOUNDATION MODELS · ORCHESTRATION

Orchestration is the decision, not the model.

The **orchestrator** is the loop between the user, the model, and the agent's capabilities. It decides what to retrieve, when to call a tool, when to ask for clarification, when to escalate, and when to stop.



PATTERNS TO KNOW

01 · Plan-act loop. Read the world, choose, observe, repeat.

02 · Handoff. When out of scope, defer to a sibling agent.

03 · Tool gating. Some actions require confirmation; pause and ask.

04 · Stop conditions. Max steps, max tokens, max retries.

05 · Escalation. When uncertain, hand to a human — not a guess.

WHERE COPILOT STUDIO HELPS

Orchestration is configured — not coded. Each control sits on the agent canvas as a setting. The runtime is provided; the strategy is yours.

AGENT TYPES · CHOOSE BY INTENT

Four ways to ship an agent.
Each trades flexibility for time-to-value.

01 · SCOPED · MINUTES TO SHIP

SharePoint agent

An agent scoped to one document library. Surface-first, very narrow, very fast.

Made for

A team's shared knowledge base; Q&A over a folder.

Constraint

One library. No custom actions. No multi-source retrieval.

02 · CONFIGURED · HOURS TO SHIP

Declarative agent

Tailored Copilot — defined by instructions, knowledge, and a small set of actions. Sits inside the existing Copilot surface.

Made for

Departments; specific outcomes inside Microsoft 365.

Constraint

No autonomous loops. Reuses Copilot's orchestrator.

03 · AUTHORED · DAYS TO SHIP

Copilot Studio agent

The full low-code canvas — topics, knowledge sources, connectors, agent flows, channels, telemetry.

Made for

Multi-source, multi-channel, governance-first back-office work.

Constraint

Tuned by maker power-users, not by end-users.

04 · LOOPING · CONTINUOUS

Autonomous agent

Triggered by events, runs multi-step, reacts to outcomes, escalates to humans when needed.

Made for

Long-running, event-driven business processes.

Constraint

Heaviest guardrails required. Demand observability first.

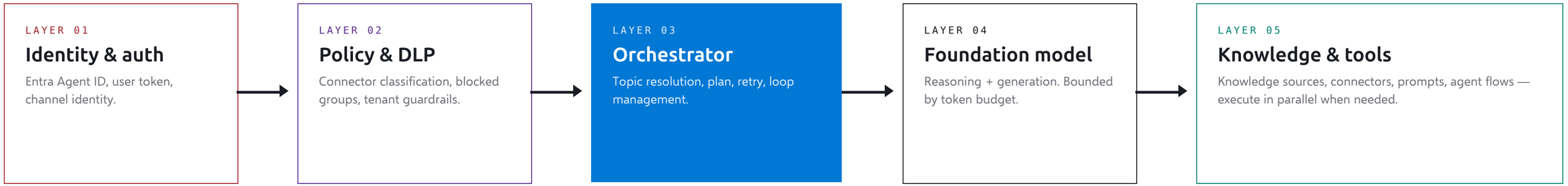
LESS COMPOSITION

MORE AUTONOMY

ARCHITECTURE · AN AGENT AT RUNTIME

A wiring diagram for the runtime, not the canvas.

At runtime, a request travels through identity, governance, orchestration, the model, and external capability — and back. Each box on the right is a layer you can audit, restrict, or scale independently.



CROSS-CUTTING

Telemetry

Per-turn traces, latency, messages, satisfaction.

CROSS-CUTTING

Audit log

Who asked, what was retrieved, what was sent back.

CROSS-CUTTING

Prompt surface

What the model actually saw — diff-able across turns.

CROSS-CUTTING

Eval & ratings

Continuous quality signal; tied to release.

MEMORY WRITE

When the answer is wrong, store the failure

Every artefact — eval set, prompt, tool, rate-limit hit — is data. The architecture's job is to make the loop close.

WHY THIS VIEW MATTERS

Most outages are not model outages. They are policy outages, permission outages, and budget outages. Architecture is the place where those become visible before the user does.

ENVIRONMENTS · WHERE AGENTS LIVE

An environment is a bounded workspace for an agent's life.

An **environment** is the storage, compute, and policy boundary for Copilot Studio artefacts. It holds the agent definition, knowledge sources, flows, connections, and the data those flows read or write — and it is where DLP policies and capacity are scoped.

01

Default

Auto-created. Shared by the tenant. Where most accidents happen.

PERSONAL · TRY-IT · DO NOT STAY

02

Developer / sandbox

Per-maker. Quick reset. Safe to break, free to experiment.

MAKER-OWNED · ISOLATED

03

Production

What the user sees. Solutions are deployed here. Locked down.

MANAGED · GOVERNED · OBSERVED

04 · WRAPS THE OTHERS

Managed Environments

A flag, not a fourth type. Adds limits, sharing controls, usage insights on top of any base environment.

PROMOTION FLOW

Sandbox → UAT → Production.

Solutions are the unit of promotion. They capture an agent, its flows, its connections, and its schema — and they move between environments as one.

DEFAULT QUOTA PER ENVIRONMENT

Database	**15 GB**
File	**20 GB**
Log	**2 GB**

Set per environment & adjustable in the Power Platform admin center.

TENANTS · BOUNDARIES & GEOGRAPHIES

A tenant is the largest trust boundary your agent inherits.

A **tenant** is the Microsoft Entra directory that owns every user, every group, every licence, every environment. Cross-tenant access is the exception, not the rule — and it is always auditable.

TENANT

contoso.onmicrosoft.com

Entra directory · billing scope · Azure region hint · 1 licence pool.

Region lock

data + compute geography

ENVIRONMENT

HR · Dev

2 hr-help-desk agents. Loose policies, real-but-copied HR data.

MEMBERS

· HR engineering (5)
· 1 HR partner reviewer

ENVIRONMENT

HR · UAT

Solution-validated with 1 release cycle. Synthetic data, real permissions.

MEMBERS

· Pilot users (40)
· QA + value-tracking

ENVIRONMENT

HR · Production

Live user traffic. DLP strict. Monitoring on. Backups taken.

MEMBERS

· Org-wide (15,400)
· Service accounts only

RULES OF THE ROAD

- **Users authenticate once, against one directory.**
- **Cross-tenant data requires explicit consent.**
- **Geography travels with the environment.**
- **Capacity and licences live at the tenant layer.**
- **Audit logs are tenant-wide.**

Plan the tenant boundary once. Re-architecting across tenants later is the most expensive move in this stack.

LICENSING · PAY FOR MESSAGES, NOT SEATS

Capacity is bought, billed, and consumed — by message.

Copilot Studio is sold as a **tenant-wide** product. You buy **Copilot Credit capacity packs**, each adding 25,000 credits per month, and each agent interaction consumes credits in proportion to the work it does.

PER PACK

25,000

credits / month · billed tenant-wide

WHERE CREDITS GO

Per-message metering

One *message* = one user turn resolving against an agent — including any tool calls or retrieval it triggered.

POOLS AT TENANT

Reallocate, don't hoard

Capacity can be reallocated at the tenant or environment level in the Power Platform admin center.

CARRIES OVER

Monthly cycle, observed

Unused capacity does not roll over indefinitely — monitor usage, alert on thresholds, plan peaks.

IMPORTANT FINE PRINT

A Microsoft 365 Copilot licence does not pool into Copilot Studio capacity.

Users with M365 Copilot get metered access for in-product usage; the tenant's pack pool is separate and reallocable.

OPERATING IMPLICATIONS

DESIGN IMPLICATION

Wide retrieval, narrow actions. Each unnecessary tool call is a credit out the door.

OPERATIONAL IMPLICATION

Alert on usage. Reallocate packs before they hit the ceiling — not the morning after.

REPORTING IMPLICATION

Reports show messages, not features. Count what you can attribute to a turned-around decision.

RESPONSIBLE AI · PRINCIPLES AS PRACTICE

Responsible AI isn't a slide at the end — it's the runtime.

Six principles name what responsible AI looks like. **Copilot Studio makes them controls**: settings on a canvas, pre-flight checks at runtime, and visible artefacts in every response.

01
Fairness
Test prompts across accents, names, dialects. Watch for disparities by department or geography.

02
Reliability & safety
When uncertain, escalate. When risky, require confirmation. Refuse unsafe prompts.

03
Privacy & security
Permissions ride with the user. Sources cite. Logs record. Boundaries hold.

04
Inclusiveness
Multi-language intents, accessible channels, screen-reader ready outputs, plain-language fallback.

05
Transparency
Cite sources. Mark AI-generated answers. Surface confidence when low. Never ghost-write.

06
Accountability
A human owns every agent. Reviews happen. Telemetry answers: who, what, when, to whom.

IN-ASK SAFEGUARDS

A guardrail flow that runs every turn



Each stage blocks a different failure mode. None of them is optional in production.

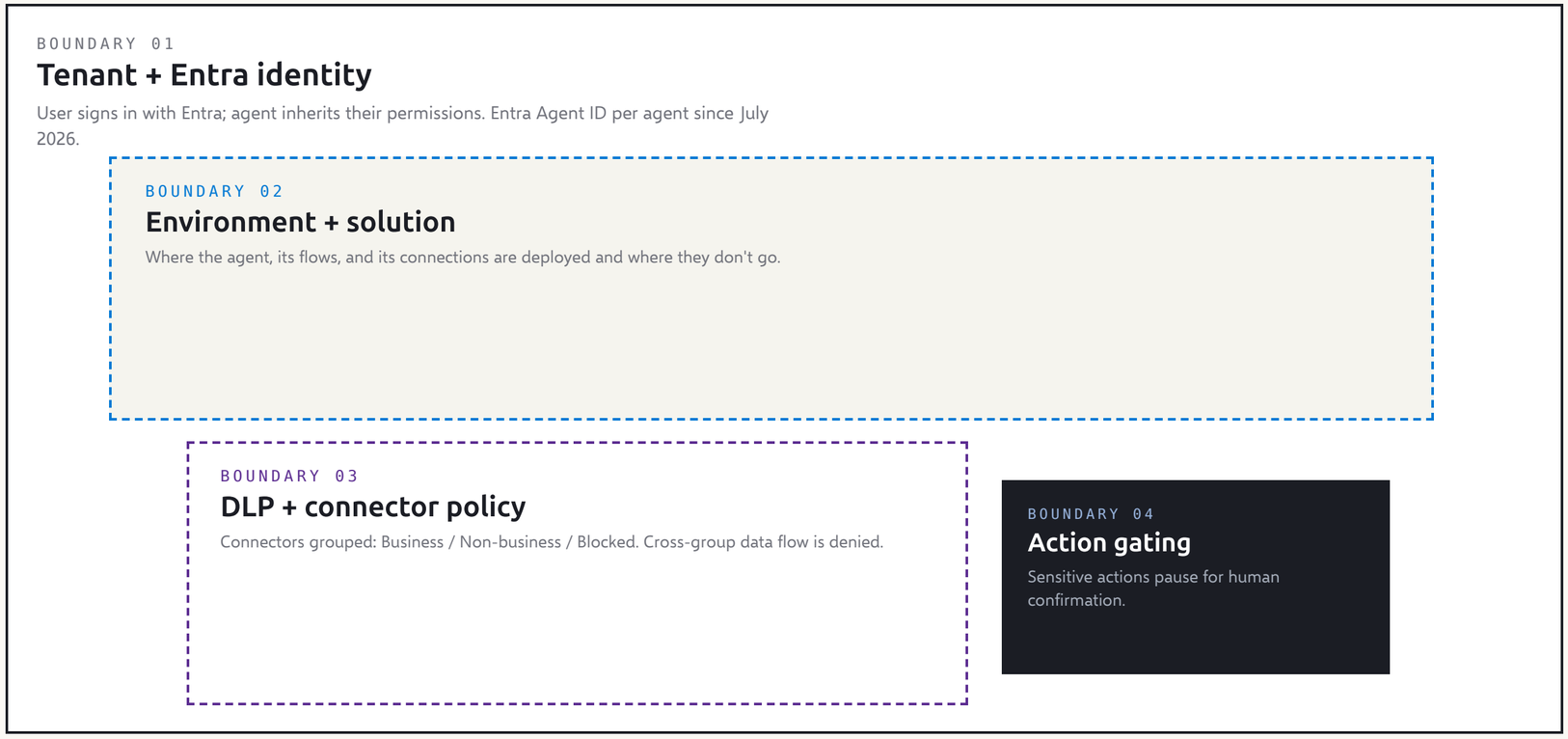
WHAT THIS LOOKS LIKE IN COPILOT STUDIO

Content moderation on input & output. Generative answers flagged as such. Source citations attached by default. Failure path routed to a human queue. Every guardrail is a setting — and an audit entry.

SECURITY CONTEXT · THE BOUNDARIES THAT HOLD

An agent is a boundary, not a feature.

Copilot Studio inherits Microsoft 365, Microsoft Entra, and Power Platform governance — and adds its own layering. The job is not to lock everything down; the job is to make every boundary **visible, configurable, and auditable**.



THE CONTROLS THAT COME FOR FREE

01 · SENSITIVE-DATA CLASSIFICATION

SharePoint scopes + Purview labels tag what the agent can see.

02 · RUNTIME CONTENT MODERATION

In/Out filters for prompt-injection and harmful content.

03 · AUDIT + TELEMETRY

Logs of who asked what, who got what, what was retrieved.

04 · ADMIN OVERRIDE

A human-in-the-loop path every agent must offer.

LAB · 1 OF 2

Provision, then describe.
Eight steps. One running agent.

Half of this lab is bureaucracy, half is authoring. The bureaucracy is where most teams stall — so we do it first, deliberately, and only once.

01

SIGN IN

Open the Power Platform admin center

`admin.powerplatform.microsoft.com`

02

PROVISION

Create a developer environment

`Manage → Environments → New · name · region`

03

ENABLE

Turn on generative AI features

`Settings → Generative AI → Allow`

04

OPEN

Switch into Copilot Studio

`copilotstudio.microsoft.com · pick env`

05

DESCRIBE

New agent — describe it in plain language

Purpose, tone, who it helps. Include "do not".

06

GROUND

Add a knowledge source

A SharePoint site or a Dataverse table. Pick one to start.

07

RESTRICT

Apply a DLP policy

Classify the SharePoint connector as Business data.

08

SAVE

Save — do NOT publish yet

Test in the Test pane before publishing. Move to lab part 2.

CHECKPOINT — PART 1

By the end of this half

— A developer environment you own.

— Generative AI enabled.

— A named agent with intent.

— One knowledge source bound.

— A DLP policy in place.

DON'T

Don't add every feature now. One knowledge source, one intent, one connector category. Earn complexity later.

LAB · 2 OF 2

Test, publish, observe. Then test again.

Publishing is the easy button. The discipline is what happens around it — the test loop, the conversation map, the failure triage, the re-publish.

STEP 09

Test in the pane

Ask the obvious, the awkward, the failure mode. Reset between turns.

STEP 10

Read the conversation map

Which topic fired. What was missed. Where retrieval failed. Pin to a list.

STEP 11

Publish one channel

Demo website first. Share with two named testers. Not the whole tenant.

STEP 12

Watch & log

Sessions, satisfaction, abandon rate. Read the unfired topics — those are the gaps.

STEP 13

Tune, then re-publish

Add one topic. Tighten one instruction. Re-test. The cycle never ends — that's the point.

LOOP · THE AGENT IS NEVER "DONE"

YOU'LL KNOW IT WORKS WHEN...

SIGNAL 01

A user re-asks the same question and gets the same answer — and the second time is faster.

SIGNAL 02

A non-trivial task moves from 30 minutes to 2 minutes for a real user.

SIGNAL 03

An admin opens analytics and names the one topic to fix next.

SIGNAL 04

A second use case is built on the first, not after it.